

Université de la Manouba
École Nationale des Sciences de l'Informatique

RAPPORT DE STAGE D'ÉTÉ III

Sujet :

**Ingestion multi-formats de relevés bancaires et
intégration d'un moteur de détection de fraude au sein
de la plateforme BankMatch**



Réalisé par :

Oussema NEHDI

Organisme d'accueil : ELITECOM

Bureau C-4, Immeuble Cléopâtre, Centre Urbain Nord, 1002 Tunis
www.elitecom.com.tn – contact@elitecom.com.tn – +216 70 033 062

Période : 7 juillet 2026 – 7 septembre 2026

Encadrant professionnel : Hedi ZOUARI

Année universitaire : 2025/2026

REMERCIEMENTS

Je remercie toutes les personnes qui m'ont accompagné pendant ce stage et qui ont contribué à la réalisation de ce projet.

À mon encadrant professionnel, Monsieur Hedi ZOUARI, je remercie pour ses conseils sur l'architecture et les revues de code. Son accompagnement a été précieux pour corriger le parser MT940, clarifier le flux JWT inter-services et mieux structurer la validation des transactions.

À l'équipe technique d'ELITECOM, je remercie pour les échanges sur les modèles de données, les tolérances de déduplication et les contraintes liées au traitement des fichiers bancaires.

À l'École Nationale des Sciences de l'Informatique (ENSI), je remercie pour la formation reçue en développement logiciel, architecture logicielle et intégration de services.

Enfin, à ma famille et à mes proches, je remercie pour leur soutien durant ce stage.

Table des matières

Liste des acronymes	v
1 Introduction	1
1.1 Contexte du stage	1
1.2 Présentation du projet	1
1.3 Périmètre du projet	2
1.4 Problématique	2
1.5 Objectifs du stage	3
2 Présentation du projet et analyse des besoins	4
2.1 Présentation de l'entreprise	4
2.2 Analyse des besoins fonctionnels	4
2.3 Besoins non fonctionnels	5
2.4 Vue d'ensemble du projet et des modules	5
2.5 Autres modules de la plateforme	6
2.6 Module de gestion de trésorerie	6
2.7 Architecture globale du système	6
2.7.1 Pipeline d'ingestion et de traitement	6
3 Architecture technique et choix de conception	9
3.1 Structure fonctionnelle du système	9
3.2 Choix d'architecture	9
3.3 Flux de traitement des données	9
3.4 Intégration des services	10
3.4.1 Intégration BankMatch	10
3.5 Langages, outils et technologies	11
3.6 Justification des choix techniques	11
3.6.1 Float vs Decimal pour les montants	11
3.6.2 Algorithme JWT HS256	12
3.6.3 Parsers maison vs bibliothèques existantes	12

4	Module Multi-Banking	13
4.1	Rôle fonctionnel	13
4.2	Formats de fichiers bancaires et de paiement pris en charge	13
4.3	Normalisation et transformation des données	14
4.4	Validation des transactions	14
4.5	Travail réalisé et enjeu technique	15
5	Module de détection de fraude	16
5.1	Rôle du moteur de fraude	16
5.2	Approche hybride	16
5.2.1	Explicabilité et interprétation des résultats	16
5.2.2	Configuration des seuils et réglage du moteur de règles	17
5.3	Règles métier et indicateurs de risque	18
5.4	Machine learning et explicabilité	19
5.4.1	Sources et données d'entraînement	20
5.5	Analyse de graphes	20
5.6	Travail réalisé	20
6	Sécurité, intégration et supervision	22
6.1	Sécurité des échanges	22
6.1.1	Limites et angles morts de sécurité	22
6.2	Intégration entre services	23
6.3	Monitoring et supervision	23
6.3.1	État actuel du monitoring	23
6.4	Déploiement et exécution	24
7	Tests, qualité et validation	25
7.1	Tests implémentés et distinction des types	25
7.2	Résultats de tests	26
8	Difficultés rencontrées et apprentissages	27
8.1	Difficultés techniques	27
8.2	Difficultés organisationnelles	28
8.3	Apprentissages personnels	28
8.4	Bilan personnel	28
9	Conclusion	29
10	Bilan et perspectives	30
10.1	Bilan du travail réalisé	30
10.2	Limites de mon intervention	30

10.3 Limites actuelles du système	31
10.4 Perspectives d'amélioration	31
11 Références	32

Table des figures

1.1	Visualisation des résultats dans l'interface globale du projet	3
2.1	Architecture globale de la plateforme SaaS d'intelligence financière avec microservices	6
2.2	Pipeline d'ingestion et de traitement implémenté	7
3.1	Flux d'intégration : phase actuelle (Fraud) et intégration prévue (BankMatch)	11
4.1	Interface d'import d'un fichier bancaire dans le module Multi-Banking . . .	14
5.1	Approche hybride combinant règles métier, machine learning et analyse de graphes	17
5.2	Analyse d'explicabilité des alertes générées par le moteur de fraude	17
5.3	Interface de configuration des seuils de détection	18
5.4	Impact de la modification des seuils sur le comportement du moteur de règles	18
5.5	Règles métier déclenchées et scores de risque associés aux transactions ana- lysées	19

Liste des acronymes

IA	Intelligence artificielle
ML	Machine Learning
API	Application Programming Interface
REST	Representational State Transfer
SaaS	Software as a Service
SHAP	SHapley Additive exPlanations
JWT	JSON Web Token
CSV	Comma-Separated Values
CAMT	Cash Management
MT940	Standard de relevé bancaire SWIFT
UI	User Interface
UX	User Experience

Chapitre 1

Introduction

1.1 Contexte du stage

J'ai effectué mon stage au sein de l'entreprise ELITECOM du 7 juillet au 7 septembre 2026. L'entreprise développe des solutions numériques pour le secteur financier, notamment des plateformes SaaS d'intelligence financière. Mon stage s'est concentré sur le développement de deux modules clés d'une plateforme de rapprochement bancaire : Multi-Banking et Fraud Detection.

Le contexte du stage était technique : j'ai travaillé sur du code Python (FastAPI) pour le backend, avec des tests en pytest, et j'ai manipulé des fichiers bancaires de test dans différents formats (CSV, CAMT.053, MT940). Pendant ces deux mois, j'ai dû construire un pipeline fiable pour traiter les transactions bancaires et détecter les anomalies, en m'intégrant à un codebase existant plutôt qu'en partant d'un projet vide.

1.2 Présentation du projet

Le projet s'inscrit dans un contexte plus large de gestion financière intelligente et de détection des anomalies sur les mouvements bancaires. Mon périmètre a porté sur les services d'ingestion et de normalisation bancaire, ainsi que sur leur intégration avec le moteur de détection de fraude. J'ai dû comprendre l'architecture existante pour situer mon travail et éviter de casser les fonctionnalités déjà en place.

La contribution personnelle peut être structurée comme suit (avec indication du statut de chaque élément) :

Le système repose sur deux modules principaux :

- **Multi-Banking** : ingestion de fichiers bancaires (CSV, CAMT.053, MT940), parsing, normalisation vers un format pivot, validation (IBAN, dates, montants, doublons), et transmission au service Fraud Detection via API REST ;
- **Fraud Detection** : analyse des transactions normalisées, application de règles métier, score de risque basé sur les modèles existants et explicabilité via SHAP.

Le rapprochement bancaire proprement dit, ainsi que l'orchestration centrale au niveau

TABLE 1.1 – Synthèse de ma contribution au projet

Élément	Contribution	Statut
CSV parser	Développé par moi	Réalisé
CAMT.053	Développé par moi	Réalisé
MT940	Développé + bug fix	Réalisé
PAIN.001	Développé	Intégration partielle à valider
Validation	Développé par moi	Réalisé
JWT	Développé par moi	Réalisé
Retry	Développé par moi	Réalisé
Tests	Développé par moi	Réalisé
Random Forest	Étudié (benchmark)	Existant
XGBoost calibré	Étudié/exploité	Existant
Isolation Forest	Étudié/exploité	Existant
SHAP	Étudié/exploité	Existant
Rules Engine	Étudié/exploité	Existant
Neo4j	Étudié/exploité	Existant
BankMatch	Modifié/intégré	Désactivé
E2E	–	Non réalisé

de la plateforme BankMatch, relèvent d'un périmètre plus large que celui de ce stage. Mon travail a donc consisté à préparer et à sécuriser le flux de données jusqu'au moteur de détection de fraude, sans prétendre développer l'ensemble de la plateforme SaaS.

1.3 Périmètre du projet

Il est important de clarifier les responsabilités au sein de la plateforme BankMatch :

- **Responsabilité de ce stage** : parsers multi-formats (CSV, CAMT.053, MT940), normalisation vers schéma pivot, validation des transactions, détection de doublons, authentification JWT inter-services, intégration avec le moteur de fraude existant.
- **Responsabilité de BankMatch** : moteur de matching (rapprochement bancaire), frontend central, orchestration globale de la plateforme, gestion multi-tenant et aspects SaaS (facturation, scalabilité).

L'intégration avec BankMatch est implémentée mais désactivée par défaut (`BANKMATCH_INTEGRATION_ENABLED=false`) en attente de la finalisation du contrat API par l'équipe BankMatch.

1.4 Problématique

Les entreprises reçoivent des relevés bancaires dans des formats hétérogènes (CSV, CAMT.053, MT940) avec des structures différentes. Par exemple, un fichier CSV peut

rendre fiable le flux d'ingestion et de normalisation bancaire et de le connecter au module de détection de fraude existant. En pratique, cela signifiait comprendre un code existant, modifier des fonctions sans casser les autres services, et m'assurer que les données transmises respectaient le format attendu par le moteur de fraude.

Chapitre 2

Présentation du projet et analyse des besoins

2.1 Présentation de l'entreprise

ELITECOM intervient dans le développement de solutions numériques destinées à accompagner les transformations du secteur financier et de la gestion d'entreprise. L'entreprise place l'innovation technologique au service des besoins liés à la gestion financière, à la sécurité des transactions et à l'exploitation des données.

Dans ce contexte, la mise en place d'un système de rapprochement bancaire et de détection de fraude répond à un besoin réel : réduire le travail manuel, sécuriser les opérations et améliorer la qualité des décisions basées sur les données financières.

2.2 Analyse des besoins fonctionnels

Le système doit traiter des fichiers bancaires dans des formats hétérogènes. Par exemple, dans le module Multi-Banking, j'ai dû développer des parsers pour :

- **CSV** : lecture avec `csv.DictReader`, extraction des colonnes `account_iban`, `value_date`, `label`, `amount`, `counterparty_iban`, `reference` ;
- **CAMT.053** : parsing XML avec `lxml.etree`, extraction des balises `<Ntry>`, `<Amt>`, `<CdtDbtInd>`, `<ValDt>`, `<AddtlNtryInf>` ;
- **MT940** : parsing avec regex, extraction des lignes `:61:` (transactions) et `:60F:` (solde).

Après parsing, les transactions sont normalisées vers le modèle `PivotTransaction` avec les champs : `tenant_id`, `bank_id`, `account_iban`, `value_date`, `label`, `amount`, `currency`, `counterparty_iban`, `reference`, `source_format`, `source_line_hash` (SHA-256).

Les besoins fonctionnels sont :

- ingérer des fichiers CSV, CAMT.053 et MT940 via l'endpoint `/banking/api/multi-banking/ingest` ;
- normaliser vers `PivotTransaction` avec hash SHA-256 pour déduplication ;

- valider : IBAN présent, date ISO 8601 valide, montant non nul, doublons détectés (tolérance ± 0.02 EUR) ;
- transmettre au service Fraud Detection via POST `/api/analyze` avec authentification JWT ;
- recevoir les résultats d'analyse : score de risque, règles déclenchées, explicabilité SHAP.

2.3 Besoins non fonctionnels

Sécurité : Les services communiquent via JWT avec le secret partagé `FRAUD_INTERNAL_SECRET`. Dans `multi-banking/main.py`, j'ai implémenté la génération du jeton avec `jwt.encode()` et sa transmission dans l'en-tête HTTP `Authorization`, selon le schéma Bearer.

Fiabilité : Le système inclut un mécanisme de retry avec backoff exponentiel (3 tentatives, délai initial 0.5s) pour les appels au service Fraud Detection. Les erreurs sont loggées avec request ID UUID pour traçabilité.

Testabilité : J'ai écrit des tests d'intégration dans `test_integration_suite.py` couvrant le pipeline complet : parsing $\rightarrow$ validation $\rightarrow$ appel fraud service (mocké avec `respx`). Les tests vérifient les cas d'erreur (fichier vide, format invalide, service indisponible).

2.4 Vue d'ensemble du projet et des modules

Le projet est une plateforme modulaire. Mon travail s'est concentré sur deux services :

Multi-Banking Service (port 8010) :

- fichiers : `main.py`, `models.py`, `validators.py` et le répertoire `parsers/` ;
- endpoints : `/banking/api/multi-banking/parse`, `/banking/api/multi-banking/validate` et `/banking/api/multi-banking/ingest` ;
- fonctionnalités : parsing multi-format, validation avec tolérance ± 0.02 EUR, transmission à Fraud Detection.

Fraud Detection Service (port 8005) :

- fichiers : `main.py`, `rules_engine.py`, `features.py`, `graph_engine.py` ;
- endpoints : `/api/analyze`, `/api/config/thresholds`, `/api/graph/*` ;
- fonctionnalités : règles métier, ML (XGBoost, Isolation Forest), SHAP, Neo4j.

Les autres modules (trésorerie, reporting, AI Financial Copilot) existent mais n'étaient pas dans mon périmètre technique. Ces modules ont été développés par d'autres membres de l'équipe et s'intègrent dans l'architecture globale de la plateforme BankMatch.

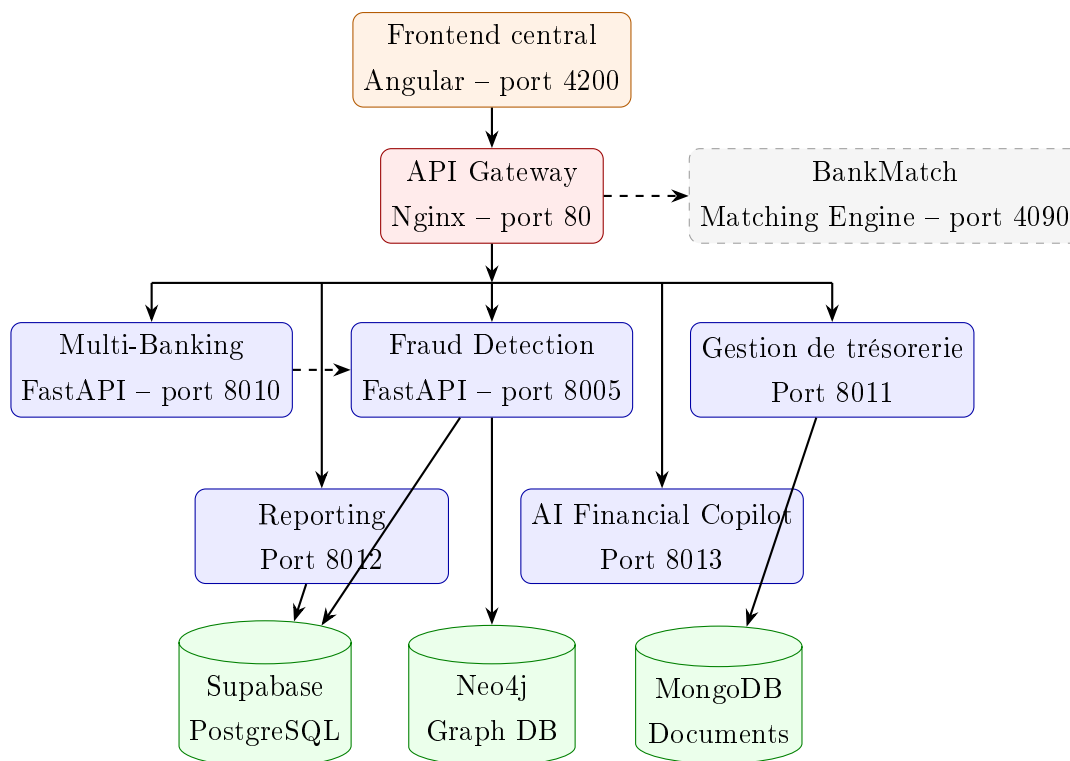


FIGURE 2.1 – Architecture globale de la plateforme SaaS d'intelligence financière avec microservices

2.5 Autres modules de la plateforme

Outre les modules Multi-Banking et Fraud Detection sur lesquels j'ai travaillé, la plateforme BankMatch comprend d'autres modules développés par l'équipe technique. Ces modules s'intègrent dans l'architecture globale pour offrir une solution complète d'intelligence financière.

2.6 Module de gestion de trésorerie

La plateforme comprend également des modules de trésorerie, de reporting et d'assistance financière. Ces modules étaient hors de mon périmètre et ne sont présentés ici que pour situer le Multi-Banking et Fraud Detection dans l'architecture globale.

2.7 Architecture globale du système

2.7.1 Pipeline d'ingestion et de traitement

Le flux de données implémenté dans mon travail est le suivant :

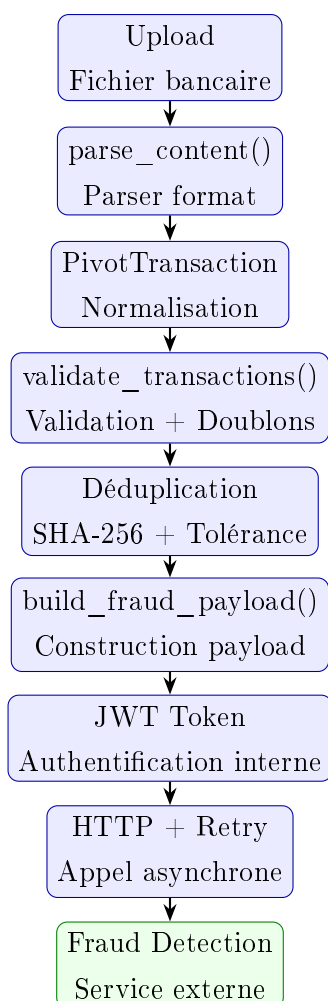


FIGURE 2.2 – Pipeline d’ingestion et de traitement implémenté

Le flux de données détaillé est le suivant :

1. Fichier bancaire (CSV/CAMT.053/MT940) téléversé via `POST/banking/api/multi-banking/ingest`;
2. Parsing dans `parse_content()` selon le format, avec retour d’une liste de `PivotTransaction`;
3. Validation dans `validate_transactions()` : vérification de l’IBAN, de la date ISO 8601, du montant et des doublons ;
4. Construction du payload Fraud Detection dans `build_fraud_payload()`, avec déduction des soldes ;
5. Génération d’un jeton JWT interne avec `jwt.encode()`, incluant le service et le locataire ;
6. Appel asynchrone à `POST /api/analyze` du service Fraud Detection avec retry exponentiel (3 tentatives) ;
7. Réception des résultats : score de risque, règles déclenchées, explicabilité SHAP ;

8. Stockage des statistiques d'upload en mémoire (`total_files`, `successful`, `failed`, `total_transactions`).

Ce flux est implémenté dans `multi-banking/main.py` lignes 361-571.

Chapitre 3

Architecture technique et choix de conception

3.1 Structure fonctionnelle du système

L'architecture est de type microservices avec :

Multi-Banking Service (Python/FastAPI, port 8010) :

- `main.py` : application FastAPI avec endpoints `parse`, `validate`, `ingest` ;
- `models.py` : Pydantic `BaseModel` `PivotTransaction` avec 18 champs (`tenant_id`, `bank_id`, `account_iban`, `value_date`, `label`, `amount`, `currency`, `counterparty_iban`, `reference`, `source_format`, `source_line_hash`, `balance_before`, `balance_after`) ;
- `validators.py` : fonction `validate_transactions()` avec tolérance ± 0.02 EUR pour doublons ;
- `parsers/` : `csv_bank.py`, `camt053.py`, `mt940.py`.

Fraud Detection Service (Python/FastAPI, port 8005) :

- `main.py` : application FastAPI avec endpoint `/api/analyze` ;
- `rules_engine.py` : règles métier (`SEUIL_REGLEMENTAIRE = 10_000.0`, `MOTCLE_SENSIBLE`, `FRACTIONNEMENT_SUSPECT`) ;
- `features.py` : extraction de features pour ML ;
- `graph_engine.py` : Neo4j pour analyse relationnelle.

3.2 Choix d'architecture

Le découpage en microservices permet de développer Multi-Banking et Fraud Detection indépendamment. Multi-Banking se concentre sur parsing/normalisation, Fraud Detection sur l'analyse de risque. La communication se fait via API REST avec authentification JWT (secret partagé `FRAUD_INTERNAL_SECRET`).

3.3 Flux de traitement des données

Le flux implémenté dans `main.py` lignes 361-571 :

1. Téléversement via `POST/banking/api/multi-banking/ingest`, avec les champs fichier, format, locataire et banque ;
2. Parsing via `parse_content()` selon format (csv, camt053, mt940, pain001) ;
3. Validation via `validate_transactions()` : vérifie IBAN, date ISO 8601, montant $\neq 0$, doublons (hash SHA-256 + tolérance ± 0.02 EUR) ;
4. Construction payload Fraud Detection via `build_fraud_payload()` : conversion vers format attendu avec déduction intelligente des soldes (`balance_before`, `balance_after`) ;
5. Génération d'un jeton JWT interne signé avec l'algorithme HS256 ;
6. Appel asynchrone de l'endpoint `POSTFRAUD_SERVICE_URL/api/analyze`, avec trois tentatives et un délai exponentiel ;
7. Réception résultats : score de risque, règles déclenchées, explicabilité SHAP ;
8. Mise à jour statistiques upload en mémoire (`total_files`, `successful`, `failed`, `total_transactions`).

3.4 Intégration des services

Les services communiquent via API REST. Multi-Banking envoie des transactions normalisées (format `PivotTransaction`) à Fraud Detection. Fraud Detection retourne des résultats avec score de risque, règles déclenchées et explicabilité SHAP. L'authentification se fait via JWT avec secret partagé.

3.4.1 Intégration BankMatch

L'intégration avec la plateforme BankMatch est implémentée via le fichier `bankmatch_client.py`. Le flux cible est :

- Multi-Banking $\rightarrow$ BankMatch $\rightarrow$ Matching Engine
- Authentication via JWT interne (`MULTI_BANKING_SERVICE_SECRET`)
- endpoint d'import : `POST/api/import` ;
- endpoint de lancement du rapprochement : `POST/reconciliation/sessions/:id/matching/start`.

Cependant, cette intégration est actuellement désactivée par défaut (`BANKMATCH_INTEGRATION_ENABLED=false`) en attente de la finalisation du contrat API par l'équipe BankMatch. Les contrats d'API ne sont pas encore finalisés, notamment le format exact des DTOs et les codes d'erreur.

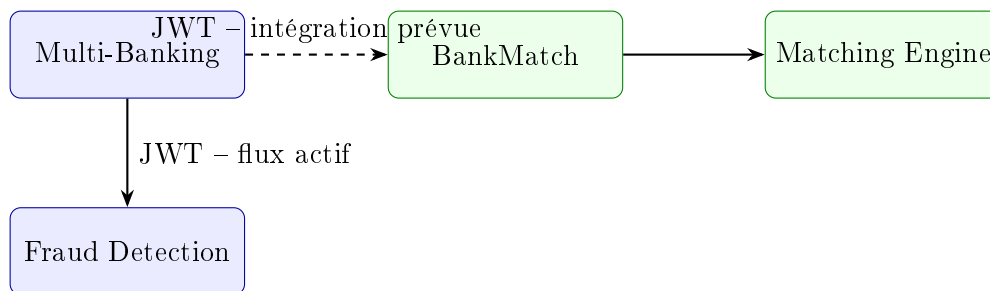


FIGURE 3.1 – Flux d'intégration : phase actuelle (Fraud) et intégration prévue (Bank-Match)

3.5 Langages, outils et technologies

Backend : Python 3.13, FastAPI [1], Pydantic, pytest, httpx (client asynchrone), jwt (authentification) [12,13], lxml (parsing XML) [11], csv (parsing CSV), re (parsing MT940 avec regex).

ML : XGBoost (modèle de classification) [9], Isolation Forest (détection d'anomalies) [10], SHAP (explicabilité) [6], joblib (sauvegarde modèles).

Base de données : Supabase [5] (PostgreSQL avec RLS), Neo4j [4] (analyse de graphes).

Le format MT940 est un standard SWIFT [7] pour l'échange de relevés bancaires, utilisé par de nombreuses banques européennes. Le format CAMT.053 est un standard ISO 20022 [8].

Déploiement : Docker [3], Docker Compose, Nginx (reverse proxy).

Monitoring : Prometheus (métriques), Grafana (visualisation), Instrumentator (FastAPI Prometheus integration).

Réglementation : Les règles de détection de fraude s'inspirent des directives PSD2 [14] et des standards FATF [15] pour la lutte contre le blanchiment et le financement du terrorisme. L'architecture globale du système est documentée dans les fichiers internes [16-19].

3.6 Justification des choix techniques

3.6.1 Float vs Decimal pour les montants

Le choix initial d'utiliser `float` pour le champ `amount` dans `PivotTransaction` s'explique par la simplicité d'implémentation et la compatibilité avec les bibliothèques ML existantes. Cependant, ce choix présente des limitations :

- Les nombres flottants peuvent introduire des erreurs d'arrondi imprévisibles

- La tolérance de ± 0.02 EUR a été introduite précisément pour absorber ces écarts de représentation
- Pour un environnement de production, l'utilisation de `Decimal` serait recommandée pour garantir la précision monétaire

La tolérance de 0.02 EUR a été retenue pour absorber les écarts de représentation flottante et les arrondis introduits par certains exports CSV. Elle expose à de faux positifs sur des paiements distincts de montants voisins ; c'est pourquoi la comparaison est restreinte aux transactions partageant IBAN, date et libellé. Une évolution consisterait à migrer `amount` vers `Decimal` et à rendre la tolérance configurable par client.

3.6.2 Algorithme JWT HS256

L'utilisation de HS256 avec secret partagé a été choisie pour simplifier le développement et les tests en environnement contrôlé. Ce choix présente des avantages :

- Simplicité de mise en œuvre pour des services internes
- Pas besoin de gestion de clés asymétriques complexes
- Adapté pour un environnement de développement mono-tenant

Cependant, pour un environnement de production multi-tenant, RS256 avec clés asymétriques serait plus adapté car :

- Meilleure séparation des responsabilités entre services
- Possibilité de révoquer des clés individuelles
- Standard de l'industrie pour les systèmes multi-tenant

3.6.3 Parsers maison vs bibliothèques existantes

Le développement de parsers personnalisés pour CSV, CAMT.053 et MT940 a été motivé par :

- Contrôle total sur le schéma pivot et les transformations
- Intégration directe avec le système de hash SHA-256 pour déduplication
- Flexibilité pour ajouter des calculs spécifiques (soldes, métadonnées)

Des bibliothèques existantes pour MT940 et CAMT n'ont pas été utilisées car elles n'offraient pas toujours la flexibilité nécessaire pour les transformations spécifiques requises par le schéma pivot du projet.

Chapitre 4

Module Multi-Banking

4.1 Rôle fonctionnel

Le module **Multi-Banking** reçoit les fichiers bancaires, les lit selon leur format, extrait les transactions et les transforme en un format exploitable par le reste du système. C'est la première étape du pipeline et elle influence directement la qualité de l'analyse finale. Pendant le stage, j'ai dû comprendre comment différents formats bancaires représentent la même information (montant, date, IBAN) et comment les normaliser vers un schéma commun.

4.2 Formats de fichiers bancaires et de paiement pris en charge

Le module a été conçu pour traiter plusieurs types de fichiers. Cette capacité est particulièrement importante dans le paysage réel de la finance, où les banques n'utilisent pas tous le même standard de transmission.

Les formats pris en charge incluent notamment :

- CSV;
- CAMT.053 (ISO 20022);
- MT940 (SWIFT);
- PAIN.001 (ISO 20022) : le projet contient un parser PAIN.001 accompagné de tests unitaires. Toutefois, contrairement aux trois formats principaux de mon périmètre (CSV, CAMT.053 et MT940), son intégration complète dans le pipeline d'ingestion reste à considérer comme partiellement validée. Le parser est chargé dynamiquement et retourne une erreur si non disponible;
- formats bancaires standardisés avec normalisation de données.

Ces différents formats peuvent présenter les mêmes informations de manière structurée différemment. Les libellés, les dates, les montants et les identifiants de compte n'ont pas toujours la même représentation. Le système doit donc normaliser ces différences pour produire un schéma unique.

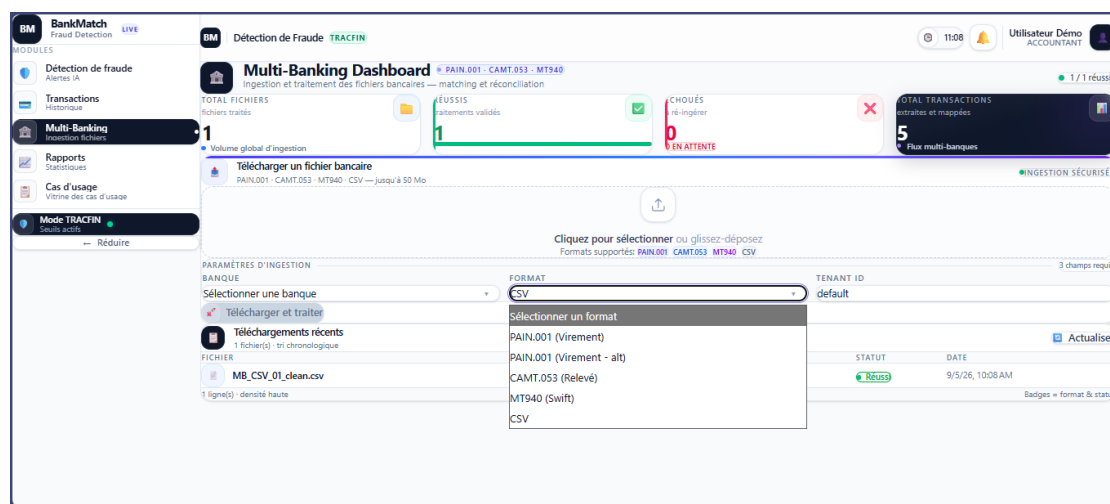


FIGURE 4.1 – Interface d'import d'un fichier bancaire dans le module Multi-Banking

4.3 Normalisation et transformation des données

Le modèle `PivotTransaction` (défini dans `models.py`) sert de format pivot :

```
class PivotTransaction(BaseModel):
    tenant_id: str
    bank_id: str
    account_iban: str
    value_date: str          # ISO 8601
    label: str
    amount: float            # signé : + crédit, - débit
    currency: str = "EUR"
    counterparty_iban: Optional[str] = None
    reference: Optional[str] = None
    source_format: str       # "csv" | "camt053" | "mt940"
    source_line_hash: Optional[str] = None
    balance_before: Optional[float] = None
    balance_after: Optional[float] = None
```

Chaque parser convertit son format vers ce modèle. Le parser CSV, défini dans `csv_bank.py`, utilise `csv.DictReader` pour créer des objets `PivotTransaction`. Le parser CAMT.053, défini dans `camt053.py`, utilise `lxml.etree` pour extraire les balises XML et calculer les soldes courants.

4.4 Validation des transactions

La fonction `validate_transactions()` dans `validators.py` implémente la validation :

- **IBAN obligatoire** : vérifie que `account_iban` n'est pas vide ;
- **Date ISO valide** : utilise `datetime.fromisoformat(tx.value_date)` ;
- **Montant non nul** : vérifie que `amount != 0` ;
- **Doublons** : détecte par hash SHA-256 exact et par tolérance ± 0.02 EUR (variable `AMOUNT_TOLERANCE = 0.02`).

La détection de doublons avec tolérance compare les montants et vérifie si l'écart est ≤ 0.02 EUR, ce qui gère les erreurs d'arrondi bancaire typiques.

4.5 Travail réalisé et enjeu technique

J'ai implémenté les parsers CSV, CAMT.053 et MT940, ainsi que la validation avec tolérance de doublons et l'intégration avec le service Fraud Detection via JWT et retry exponentiel. Dans le parser MT940, le groupe générique `\w` de la regex originale acceptait les chiffres dans le "funds code" et supprimait ainsi le premier chiffre du montant. La correction restreint explicitement ce groupe aux lettres :

Originale : `r":61:(\d{6})(\d{4})?([CD])(\w)?([\d,]+)"`

Corrigée : `r":61:(\d{6})(\d{4})?([CD])([A-Za-z])?([\d,]+)"`

Chapitre 5

Module de détection de fraude

5.1 Rôle du moteur de fraude

Le module **Fraud Detection** reçoit les transactions normalisées et vérifiées par le module Multi-Banking. Son rôle est d'évaluer le risque associé à chaque mouvement et d'identifier les opérations suspectes ou anormales.

Le système combine plusieurs niveaux d'analyse : règles métier, calcul de seuils, et un modèle ML (XGBoost) déjà présent dans le projet. Sur ce module, ma contribution a été surtout d'intégrer et de sécuriser le flux d'entrée, de clarifier le format des échanges, et de vérifier le comportement des composants déjà existants. Les moteurs de règles, le modèle de scoring et la logique de fusion ont été étudiés et exploités dans le cadre de la chaîne de traitement, sans constituer une création complète de l'architecture ML du projet.

5.2 Approche hybride

L'approche retenue est hybride. Elle associe des règles métier, des indicateurs calculés à partir des données, ainsi que des éléments d'analyse plus avancés. Cette combinaison est bien adaptée au domaine bancaire, car certains cas de fraude sont très simples à identifier à l'aide d'un seuil, tandis que d'autres nécessitent un raisonnement plus complexe sur l'historique du compte ou sur les relations entre comptes.

Dans le projet étudié, les règles métier permettent de traiter rapidement certains cas déterministes, tandis que le score ML apporte une information complémentaire sur des comportements moins explicites. Mon travail n'a pas porté sur la conception de ces modèles, mais sur l'intégration des transactions normalisées dans cette chaîne d'analyse.

5.2.1 Explicabilité et interprétation des résultats

Dans le domaine bancaire, il est essentiel de pouvoir expliquer pourquoi une alerte est déclenchée. Une transaction classée comme suspecte doit être accompagnée d'une justification technique compréhensible par les analystes.

Dans ce projet, l'explicabilité est implémentée via SHAP [6], qui permet d'identifier les variables qui influencent le plus la décision du modèle : montant, comportement du compte,

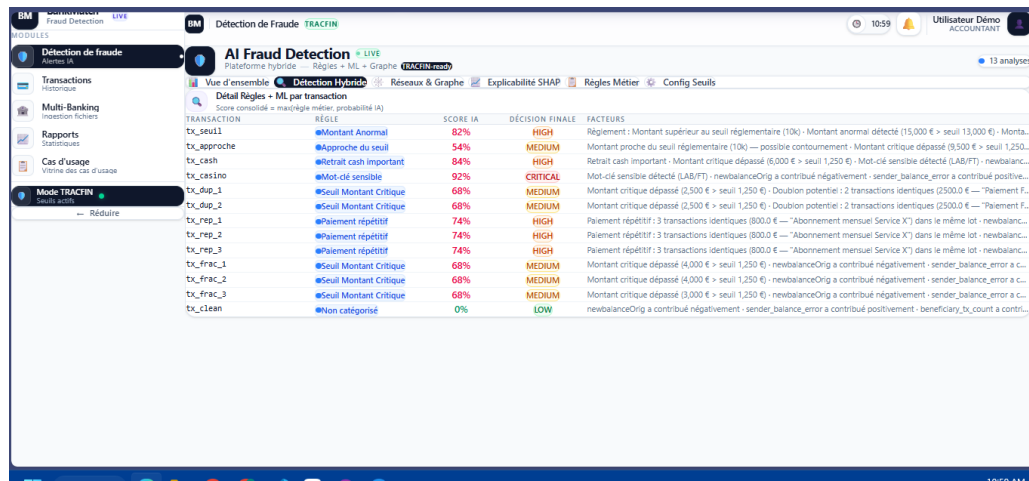


FIGURE 5.1 – Approche hybride combinant règles métier, machine learning et analyse de graphes

fréquence des transactions, ou changement inhabituel dans les habitudes du client.

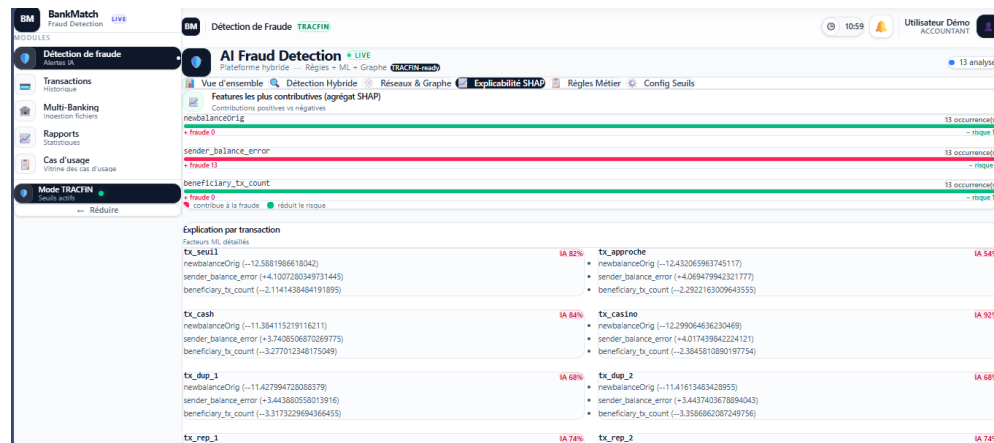


FIGURE 5.2 – Analyse d'explicabilité des alertes générées par le moteur de fraude

5.2.2 Configuration des seuils et réglage du moteur de règles

Le système n'est pas figé. Il intègre un mécanisme de configuration des seuils permettant d'ajuster le moteur de fraude selon les exigences du contexte métier. Cela est particulièrement utile quand on veut moduler le niveau de sensibilité du système en fonction des règles réglementaires ou des habitudes de l'entreprise.

Pendant le stage, j'ai également étudié la partie de configuration des seuils du moteur de règles. Cette possibilité permet de modifier certains paramètres sans changer directement le code des règles. Elle m'a surtout aidé à comprendre comment les contraintes métier étaient traduites en paramètres techniques dans le projet.

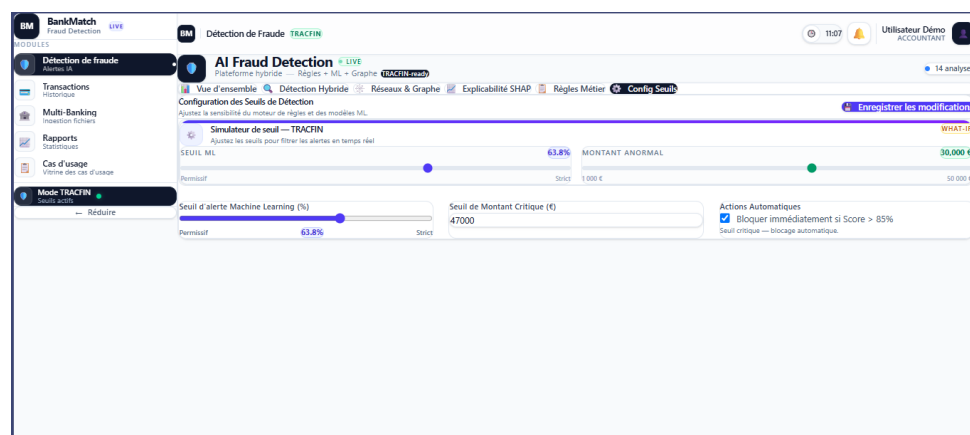


FIGURE 5.3 – Interface de configuration des seuils de détection

</

FIGURE 5.4 – Impact de la modification des seuils sur le comportement du moteur de règles

5.3 Règles métier et indicateurs de risque

Le moteur de fraude applique un ensemble de règles conçues pour détecter des situations inhabituelles. Parmi les éléments pris en compte, on trouve :

- montant supérieur à un seuil réglementaire ;
- montant proche du seuil de déclenchement ;
- retrait cash inhabituel ;
- motifs ou mots-clés sensibles dans la description ;
- transaction répétitive ou fractionnée ;
- changement de bénéficiaire ou d'IBAN inhabituel ;
- compte peu utilisé réactivé après une longue période ;
- hausse anormale du montant par rapport à la moyenne du compte.

Ces règles ne sont pas seulement descriptives. Elles vont servir à calculer un score global et à hiérarchiser les alertes selon leur gravité. C'est une étape importante pour ne pas confondre analyse générale et signalement prioritaire.

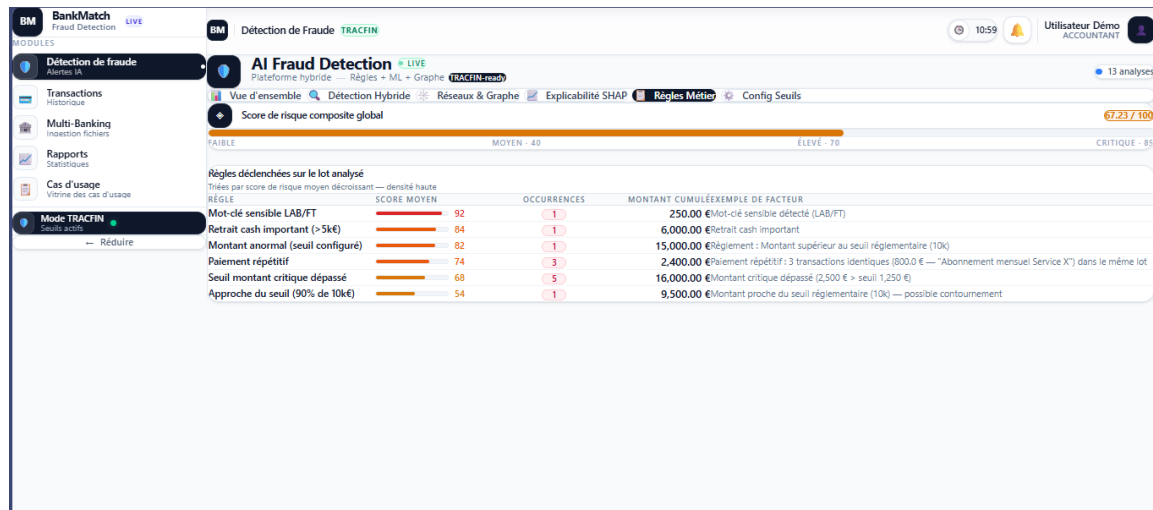


FIGURE 5.5 – Règles métier déclenchées et scores de risque associés aux transactions analysées

5.4 Machine learning et explicabilité

Le système de détection de fraude combine plusieurs composants ML qui étaient déjà présents dans le projet au début de mon stage. Mon travail sur cette partie a consisté à observer et comprendre l'implémentation existante, à étudier le fonctionnement des différents modèles et à m'assurer que les données transmises depuis Multi-Banking respectaient le format attendu.

5.4.1 Pipeline de chargement des modèles

Le fichier `main.py` (ligne 270) implémente une logique de priorité pour le chargement des modèles :

```
MODEL_PATH = "model_fraud_calibrated.pkl" if os.path.exists("model_fraud_calibrated.pkl")
else "model_fraud.pkl"
```

Cela signifie que :

- Si `model_fraud_calibrated.pkl` existe, il est chargé en priorité (XGBoost calibré)
- Sinon, le système utilise `model_fraud.pkl` comme fallback

Cette logique a été observée dans le code existant et n'a pas été modifiée pendant le stage.

5.4.2 Random Forest : modèle de référence

Un modèle Random Forest a été utilisé comme benchmark pour la détection de fraude. Implémenté avec `RandomForestClassifier` dans `train_calibrated_model.py` :

- `n_estimators=150`
- `max_depth=8`
- `min_samples_leaf=3`

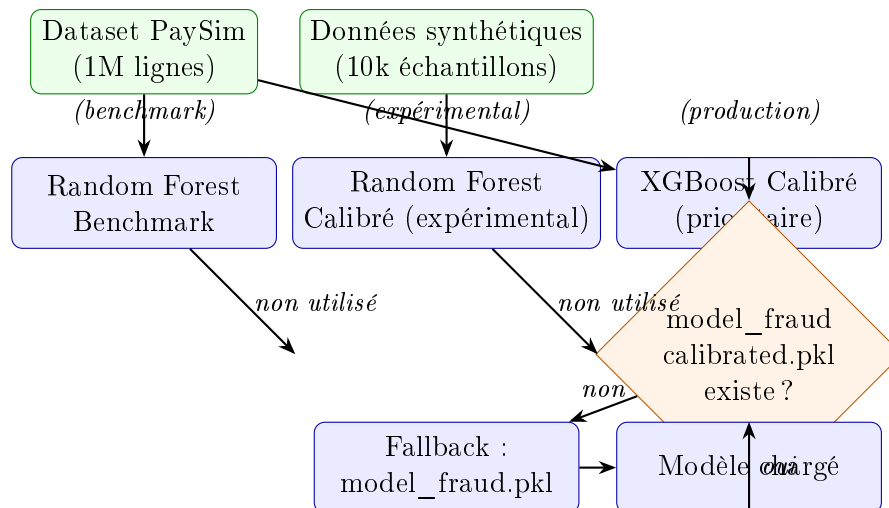


FIGURE 5.6 – Pipeline de chargement des modèles ML avec logique de priorité

- `class_weight="balanced"` pour gérer le déséquilibre de classes
- `random_state=42` pour la reproductibilité
- `n_jobs=-1` pour utiliser tous les cœurs CPU

Ce modèle a servi de référence pour comparer les performances avec XGBoost.

5.4.3 Random Forest calibré

Une expérimentation de calibration a été menée sur le Random Forest avec `CalibratedClassifierCV` dans `train_calibrated_model.py` :

- Méthode de calibration : isotone (calibration monotone)
- Cross-validation : `cv=5`
- Données utilisées : données synthétiques générées par `generate_synthetic_data()`

Cette expérimentation est distincte du Random Forest benchmark et utilise des données synthétiques plutôt que le dataset PaySim. Elle a pour objectif d'améliorer la fiabilité des probabilités prédites par le modèle.

5.4.4 XGBoost calibré

Le modèle actuellement prioritaire dans le système est un XGBoost calibré, stocké dans `model_fraud_calibrated.pkl`. Selon `train_xgboost_calibrated.py`, il utilise les paramètres suivants :

- `n_estimators=300`
- `max_depth=6`
- `learning_rate=0.1`
- `subsample=0.8`
- `colsample_bytree=0.8`
- `scale_pos_weight` : calculé automatiquement pour gérer le déséquilibre de classes

- `random_state=42`
- `n_jobs=-1`

La calibration est effectuée avec `CalibratedClassifierCV` :

- Méthode : isotone
- Cross-validation : `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`

Le seuil optimal (0.4758) est enregistré dans `model_metadata.pkl` et déterminé via maximisation du F1-score sur la courbe Precision-Recall.

5.4.5 Isolation Forest

Le système utilise également Isolation Forest, un algorithme de détection d'anomalies non supervisé [10]. Contrairement aux modèles de classification supervisée (Random Forest, XGBoost), Isolation Forest ne nécessite pas d'étiquettes de fraude et détecte des comportements atypiques basés sur l'isolation des instances dans l'espace des features.

5.4.6 SHAP : explicabilité

L'explicabilité repose sur SHAP (SHapley Additive exPlanations) [6], une méthode d'interprétation des modèles ML. SHAP n'est pas un modèle de prédiction mais une technique d'explicabilité qui identifie les variables qui influencent le plus la décision du modèle : montant, comportement du compte, fréquence des transactions, ou changement inhabituel dans les habitudes du client.

5.4.7 Rules Engine : règles métier

Le moteur de règles (`rules_engine.py`) applique des règles métier déterministes, distinctes du Machine Learning. Ces règles sont basées sur des seuils et des mots-clés définis par l'équipe métier :

- **SEUIL_REGLEMENTAIRE** : montant > 10_000.0 EUR (poids 100)
- **MOTCLE_SENSIBLE** : détection de mots-clés comme CASINO, PARIS, POKER (poids 100)
- **FRACTIONNEMENT_SUSPECT** : détection de paiements fractionnés (poids 90)
- **RETRAIT_CASH_IMPORTANT** : retrait cash > 5_000.0 EUR (poids 80)
- **HORAIRE_ATYPIQUE** : transaction entre 1h et 5h du matin (poids 25)
- **VELOCITE_ANORMALE** : >3 transactions en 15 minutes (poids 40)

Ces règles sont configurables via l'endpoint `/api/config/thresholds`.

5.4.8 Fusion des scores

Le système fusionne les scores de différentes sources via la fonction `fuse_scores()` (lignes 318 à 340 de `main.py`) :

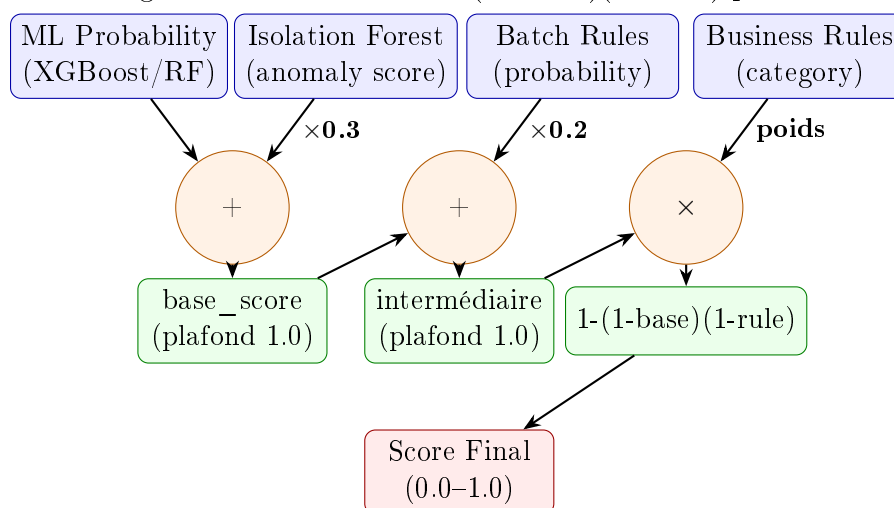
```

base_score = ml_probability
if isolation_anomaly_score > 0:
    base_score = base_score + (isolation_anomaly_score * 0.3)
if batch_probability > 0:
    base_score = base_score + (batch_probability * 0.2)
if is_blocked and rule_category:
    rule_score = RULE_SEVERITY_WEIGHTS.get(rule_category, 0.70)
    return round(1.0 - ((1.0 - base_score) * (1.0 - rule_score)), 4)

```

La logique de fusion est la suivante :

- Score de base : probabilité ML (XGBoost calibré ou Random Forest)
- Intégration Isolation Forest : +30% du score d'anomalie (plafonné à 1.0)
- Intégration batch rules : +20% de la probabilité batch (plafonné à 1.0)
- Fusion avec règles métier : formule $1 - (1 - \text{base})(1 - \text{rule})$ pour combiner les scores



Formule finale : $\text{round}(1.0 - ((1.0 - \text{base_score}) \times (1.0 - \text{rule_score})), 4)$

FIGURE 5.7 – Fusion des scores ML, Isolation Forest, Batch Rules et Règles métier

5.4.9 Variables du modèle

Les variables du modèle sont extraites dans `features.py`, notamment :

- `amount`, `oldbalanceOrig`, `newbalanceOrig`, `oldbalanceDest`, `newbalanceDest`
- `sender_balance_error` et `receiver_balance_error`
- `is_transfer` et `is_cash_out`
- `hour_of_day`, `amount_to_avg_ratio`, `days_since_last_tx`, `beneficiary_tx_count`

5.4.10 Sources et données d'entraînement

Les modèles ML du projet utilisent différentes sources de données :

Dataset PaySim : Utilisé pour l’entraînement XGBoost calibré (`train_xgboost_calibrated.py`, ligne 28). PaySim est un dataset synthétique de transactions bancaires généré par simulation [14]. Le script charge jusqu’à 1 million de lignes depuis `data/paysim.csv`.

Données synthétiques : Utilisées pour l’entraînement Random Forest calibré (`train_calibrated_model.py`, lignes 10-52). La fonction `generate_synthetic_data()` crée 10 000 échantillons basés sur les 9 variables de `features.py`.

Données de test : Les fichiers `sample.csv` et `sample_camt.xml` dans `multi-banking/data/` sont utilisés pour les tests de parsing et d’intégration.

Limites des informations disponibles : Les métriques précises d’entraînement (précision, rappel, AUC-PR) et les caractéristiques détaillées du jeu de données d’entraînement n’étaient pas toutes disponibles dans la documentation du projet. Ces éléments seraient nécessaires pour une évaluation complète de la performance du modèle.

5.5 Analyse de graphes

Le système inclut un moteur de graphes avec Neo4j (fichier `graph_engine.py`). La connexion est établie au démarrage dans `main.py` lignes 55-64. Si Neo4j n’est pas disponible, le système fonctionne sans analyse de graphe (message d’avertissement dans les logs).

L’analyse de graphes permet d’étudier les relations entre comptes, bénéficiaires et transactions pour détecter des schémas complexes comme des flux circulaires ou des comptes mules.

5.6 Travail réalisé

J’ai observé l’implémentation du moteur de règles dans `rules_engine.py` (391 lignes). Les règles incluent :

- **SEUIL_REGLEMENTAIRE** : montant > 10_000.0 EUR (poids 100) ;
- **MOTCLE_SENSIBLE** : détection de mots-clés comme CASINO, PARIS, POKER (poids 100) ;
- **FRACTIONNEMENT_SUSPECT** : détection de paiements fractionnés pour contourner le seuil (poids 90) ;
- **RETRAIT_CASH_IMPORTANT** : retrait cash > 5_000.0 EUR (poids 80) ;
- **HORAIRE_ATYPIQUE** : transaction entre 1h et 5h du matin (poids 25) ;
- **VELOCITE_ANORMALE** : >3 transactions en 15 minutes (poids 40).

Les seuils sont configurables via l’endpoint `/api/config/thresholds` et stockés dans `config_store.py`.

Chapitre 6

Sécurité, intégration et supervision

6.1 Sécurité des échanges

L'authentification inter-services est implémentée avec JWT. Dans `multi-banking/main.py`, le jeton est généré ainsi :

```
fraud_token_payload = {
    "service": "multi-banking",
    "type": "internal",
    "tenant_id": tenant_id,
}
fraud_token = jwt.encode(
    fraud_token_payload,
    FRAUD_INTERNAL_SECRET,
    algorithm="HS256",
)
```

Le secret partagé est défini dans `.env` : `FRAUD_INTERNAL_SECRET`. En développement, l'authentification peut être désactivée avec `DISABLE_INTERNAL_AUTH=true` (jamais en production).

6.1.1 Limites et angles morts de sécurité

Isolation multi-tenant : Le `tenant_id` est actuellement fourni par le client dans le formulaire puis inclus dans le JWT. Pour une véritable isolation multi-tenant, ce `tenant_id` devrait être validé par un service d'authentification central (BankMatch) avant d'être inclus dans le token interne.

Gestion des secrets : Selon `TESTING_FINDINGS_AND_RECOMMENDATIONS.md`, les secrets JWT actuels font 19-24 octets, en dessous des 32+ octets recommandés par la RFC 7518. Des valeurs par défaut comme `internal_dev_secret` et `password_super_securise` apparaissent dans des documents publics, ce qui constitue une pratique à corriger en production.

Expiration des tokens : L'expiration des tokens JWT n'est pas explicitement configurée

dans le rapport. Selon `ARCHITECTURE_DECISIONS.md`, les tokens internes ont une durée de vie de 30 minutes en développement.

Parsing XML : L'utilisation de `lxml` pour le parsing des fichiers CAMT.053 expose potentiellement à des attaques XXE (XML External Entity) et à l'expansion d'entités si non configuré correctement. Une sécurisation supplémentaire avec `defusedxml` serait recommandée.

Choix de l'algorithme JWT : L'utilisation de HS256 avec secret partagé simplifie le développement mais RS256 avec clés asymétriques serait plus adapté pour un environnement de production multi-tenant.

6.2 Intégration entre services

Les services communiquent via API REST. Multi-Banking appelle Fraud Detection avec `httpx.AsyncClient()` et un mécanisme de retry exponentiel : trois tentatives sont effectuées avec un délai initial de 0.5 seconde, doublé après chaque échec. Les codes 502, 503 et 504 déclenchent une nouvelle tentative ; les autres erreurs retournent une erreur 502 au client.

6.3 Monitoring et supervision

Le logging est structuré avec JSON dans `main.py` lignes 86-97. Chaque requête génère un log avec `request_id` (UUID), `method`, `path`, `status_code`, `duration_ms`, `environment`. Prometheus est intégré via `Instrumentator().instrument(app).expose(app)` pour exposer les métriques sur `/metrics`.

6.3.1 État actuel du monitoring

Selon le rapport de test du 19 août 2026 :

- **Prometheus** : Fonctionnel, scraping configuré pour fraud-service (port 8005) et multi-banking (port 8010) - status UP
- **Grafana** : Présent mais non fonctionnel au 19/08 (erreur "failed to load its application files")
- **Neo4j** : Base de données fonctionnelle mais connexion échouée depuis le fraud-service (`database_connected: false`)

Certaines fonctionnalités de monitoring sont marquées comme "recommandées" dans `ARCHITECTURE_DECISIONS.md` et non encore pleinement intégrées en production.

6.4 Déploiement et exécution

Les services sont containerisés avec Docker. Multi-Banking utilise le fichier `Dockerfile` (lignes 1-12) avec Python 3.13-slim et HEALTHCHECK. Le démarrage se fait avec `uvicorn main:app -host 0.0.0.0 -port 8010`.

Chapitre 7

Tests, qualité et validation

7.1 Tests implémentés et distinction des types

J'ai écrit des tests d'intégration dans `test_integration_suite.py`. Les tests couvrent :

Section 1 - CSV Parser (lignes 120-161) :

- `test_parse_csv_basic` : parsing CSV avec 2 transactions, vérifie `amount = 150.0` et `-500.0` ;
- `test_parse_csv_preserves_iban` : vérifie que l'IBAN est correctement assigné ;
- `test_parse_csv_generates_hash` : vérifie que le hash SHA-256 est généré (64 caractères) ;
- `test_parse_csv_with_balance_fields` : parsing avec les colonnes de solde avant et après transaction.

Section 2 - CAMT.053 Parser (lignes 167-199) :

- `test_parse_camt053_basic` : parsing XML CAMT.053, vérifie extraction de 1 transaction avec `amount = 2500.0` ;
- `test_parse_camt053_preserves_iban` : vérifie extraction IBAN depuis XML.

Section 3 - MT940 Parser (lignes 205-220) :

- `test_parse_mt940_basic` : parsing MT940 avec regex corrigée.

Section 4 - PAIN.001 Parser :

- 5 tests unitaires PAIN.001 présents dans le dépôt (parsers PAIN.001 pour paiements SEPA)
- Parser implémenté mais intégration complète à valider

Section 4 - Validation Rules (lignes 226-302) :

- `test_valid_transaction_passes` : transaction valide sans erreurs ;
- `test_missing_iban_fails` : erreur si IBAN vide ;
- `test_invalid_date_format_fails` : erreur si date non ISO 8601 ;
- `test_zero_amount_fails` : erreur si `montant = 0` ;
- `test_exact_duplicate_detected` : détection doublon dans même batch ;
- `test_historical_duplicate_detected` : détection doublon dans historique ;
- `test_near_duplicate_within_tolerance` : détection avec tolérance ± 0.02 EUR ;

- `test_near_duplicate_outside_tolerance` : pas de détection si écart > 0.02 EUR.

Section 6 - Ingest Pipeline Integration (lignes 334-458) :

- `test_successful_ingest_returns_results` : pipeline d'intégration simulé avec mock fraud service, vérifie response 200 et `parsed_count = 21` ;
- `test_ingest_502_when_fraud_service_errors` : gestion erreur 502 si fraud service indisponible ;
- `test_ingest_with_bankmatch_disabled` : vérifie un résultat nul lorsque l'intégration BankMatch est désactivée.

Les tests utilisent `respx` pour mocker les appels HTTP au service Fraud Detection et `pytest` pour l'exécution. La couverture de tests est mesurée avec `pytest-cov`.

7.2 Résultats de tests

Selon le document `TESTING_FINDINGS_AND_RECOMMENDATIONS.md` du 18 août 2026, les résultats de tests sont les suivants :

TABLE 7.1 – Synthèse des résultats de tests

Module	Tests	Durée	Statut	Type
Multi-Banking Backend	28	10.63s	PASSED	Unitaires + Intégration (mocks)
Fraud Detection Backend	71	243.18s	PASSED	Unitaires + Intégration
Frontend-Backend Integration	35	24.61s	PASSED	Intégration (mocks)
Total	134	278.42s	ALL PASSED	

Couverture par module :

- Multi-Banking : CSV parser (2 tests), CAMT.053 parser (6 tests), MT940 parser (6 tests), PAIN.001 parser (5 tests), Duplicate tolerance (6 tests), Ingestion integration (4 tests avec mock)
- Fraud Detection : Preprocessing features (9 tests), Rules engine (16 tests), Two-factor authentication (9 tests), Graph engine (6 tests), ML fusion (4 tests), Persistence (4 tests), Authentication (4 tests), API endpoints (19 tests)
- Frontend Integration : Multi-banking service (10 tests), Reports service (13 tests), Fraud alerts service (12 tests)

7.2.1 Analyse de la valeur des tests

Au-delà du simple décompte, les tests démontrent des propriétés importantes du système :

Limites des tests :

TABLE 7.2 – Ce que les tests démontrent

Catégorie de tests	Propriété démontrée
CSV parser	Capacité d'extraction des colonnes account_iban, value_date, label, amount
CAMT.053 parser	Gestion de la structure XML hiérarchique et extraction des balises ISO 20022
MT940 parser	Robustesse du parsing avec regex (validation du bug fix montant)
PAIN.001 parser	Support du format de paiement SEPA (intégration partielle)
Duplicate tolerance	Règle métier de détection $\pm 0.02\text{€}$ pour écarts d'arrondi
Validation IBAN	Vérification de la présence obligatoire de l'IBAN
Validation date	Conformité au format ISO 8601
Validation montant	Rejet des montants nuls
Fraud mock Service indisponible	Respect du contrat d'intégration Multi-Banking → Fraud Detection Gestion d'erreur 502 lors d'appel service défaillant
BankMatch disabled	Comportement attendu lorsque l'intégration est désactivée
Rules engine	Application correcte des seuils et mots-clés sensibles
ML fusion	Combinaison appropriée des scores ML, Isolation Forest et règles métier
Authentication	Validation des tokens JWT internes
Graph engine	Connexion Neo4j et requêtes de graphe basiques

- Les tests d'intégration mockent le service Fraud Detection (via respx) et ne constituent pas des tests end-to-end réels
- Pas de tests end-to-end avec les services réels démarrés
- Tests de retry exponentiel non couverts
- CI/CD et benchmarks non implémentés (marqués "en attente" dans le dépôt)

Chapitre 8

Difficultés rencontrées et apprentissages

8.1 Difficultés techniques

Bug du parser MT940 : Le groupe générique `\w` de la regex originale acceptait les chiffres dans le "funds code" et faisait perdre le premier chiffre du montant. Le test `test_parse_mt940_basic` échouait pour un montant attendu de 1500.00, interprété comme 500.00. La correction restreint ce groupe aux lettres. Ce bug est documenté dans `mt940.py`, lignes 1 à 8.

TABLE 8.1 – Correction du bug MT940

Élément	Avant	Après
Regex	<code>\w</code>	<code>[A-Za-z]</code>
Problème	Accepte les chiffres	Accepte uniquement les lettres
Exemple	1500.00 → 500.00	1500.00 → 1500.00
Cause	Groupe funds code trop permissif	Expression contrainte
Validation	Test parser	Test automatisé

Intégration JWT : La configuration de l'authentification inter-services a nécessité de comprendre le flux Multi-Banking → Fraud Detection. J'ai dû configurer les variables d'environnement `FRAUD_INTERNAL_SECRET` et `DISABLE_INTERNAL_AUTH`, et implémenter la génération de token avec `jwt.encode()`. En développement, l'authentification peut être désactivée, mais j'ai appris que cela ne doit jamais être activé en production.

Tolérance de doublons : La détection de doublons avec tolérance ± 0.02 EUR pour gérer les erreurs d'arrondi bancaire a été complexe à implémenter. J'ai dû comparer les montants avec une tolérance et vérifier le hash de base sans le montant pour détecter les transactions similaires. Le test `test_near_duplicate_within_tolerance` valide ce comportement.

8.2 Difficultés organisationnelles

Le projet impliquait plusieurs modules et plusieurs développeurs. J'ai dû comprendre l'architecture globale pour situer mon travail. Le document `ARCHITECTURE_DECISIONS.md` a facilité la compréhension des décisions d'architecture et de l'intégration avec BankMatch.

8.3 Apprentissages personnels

J'ai appris à développer des parsers pour des formats bancaires réels (CSV, CAMT.053, MT940), à implémenter des validations avec tolérances, à configurer l'authentification JWT entre services, et à écrire des tests d'intégration avec `pytest` et `respx`. J'ai aussi découvert l'importance de la qualité des données : une donnée mal validée peut fausser les résultats de fraude.

Avant le stage, je connaissais Python et FastAPI de manière théorique. Maintenant, je sais les utiliser dans un contexte réel avec authentification, retry exponentiel, logging structuré et monitoring Prometheus. Ce qui m'a particulièrement marqué, c'est la différence entre développer un projet à partir de zéro et intervenir sur un code existant : il faut comprendre les conventions déjà en place, ne pas casser les fonctionnalités des autres services, et documenter ses propres choix pour que les autres développeurs puissent comprendre le raisonnement.

8.4 Bilan personnel

En travaillant sur un système financier concret et en manipulant des fichiers bancaires de test représentatifs de situations rencontrées dans ce type de projet, j'ai développé des compétences techniques en parsing de fichiers, validation avec tolérances, authentification JWT, tests d'intégration et monitoring. J'ai aussi appris l'importance de la documentation technique et de la communication avec l'équipe.

Chapitre 9

Conclusion

Pendant ce stage, j'ai développé les parsers pour les formats bancaires CSV, CAMT.053 et MT940 dans le module Multi-Banking. J'ai implémenté la validation des transactions avec détection de doublons (tolérance ± 0.02 EUR) et configuré l'authentification JWT entre Multi-Banking et Fraud Detection. J'ai aussi corrigé un bug important dans le parser MT940 où la regex faisait perdre le premier chiffre du montant.

Le travail technique réalisé inclut :

- 3 parsers (`csv_bank.py`, `camt053.py`, `mt940.py`) ;
- validation avec tolérance dans `validators.py` ;
- intégration JWT et retry exponentiel dans `main.py` ;
- tests d'intégration dans `test_integration_suite.py` couvrant le pipeline complet.

En manipulant des données bancaires de test, en configurant une authentification inter-services et en écrivant des tests avec `pytest` et `respx`, j'ai pu passer de la théorie à la pratique. J'ai compris que la qualité des données est essentielle pour un système de détection de fraude.

Chapitre 10

Bilan et perspectives

10.1 Bilan du travail réalisé

Le travail effectué a permis de développer et de stabiliser le flux d'ingestion bancaire dans le module Multi-Banking, puis de le relier au moteur de détection de fraude. La contribution principale a porté sur les parseurs, la validation des transactions, la génération du token interne JWT et la vérification du pipeline d'intégration. Les parties liées à la détection de fraude et aux modèles ML ont été étudiées et intégrées dans le système existant, sans constituer l'ensemble d'une architecture de conception originale de ce moteur.

10.2 Limites de mon intervention

Il est important de préciser clairement les limites de mon intervention dans ce projet :

- **Modèles ML préexistants** : Les modèles XGBoost, Isolation Forest et l'explicabilité SHAP étaient déjà présents dans le projet avant mon stage. Mon travail a consisté à les étudier, à comprendre leur fonctionnement et à intégrer les transactions normalisées dans ce pipeline existant.
- **Neo4j préexistant** : L'infrastructure Neo4j et le moteur de graphes étaient déjà implémentés. J'ai observé et documenté leur fonctionnement sans participer à leur conception.
- **BankMatch non activé** : L'intégration avec la plateforme BankMatch est implémentée dans le code mais désactivée par défaut (`BANKMATCH_INTEGRATION_ENABLED=false`) en attente de la finalisation du contrat API par l'équipe BankMatch.
- **Tests E2E non réalisés** : Les tests d'intégration utilisent des mocks (respx) pour simuler le service Fraud Detection. Un véritable test end-to-end avec les services réels n'a pas été réalisé.
- **CI/CD non implémenté** : L'infrastructure CI/CD et les benchmarks de performance ne sont pas encore implémentés (marqués "en attente" dans le dépôt).

10.3 Limites actuelles du système

Le système présente plusieurs limites techniques et fonctionnelles qu'il convient de signaler de manière honnête :

10.3.1 Limites des données et modèles

- **Données synthétiques** : Les modèles ML ont été entraînés sur des données synthétiques (PaySim, données générées par `generate_synthetic_data()`). Ces données ne reflètent pas nécessairement les patterns de fraude réels et peuvent limiter la généralisation du modèle en production.
- **Métriques d'entraînement** : Les métriques précises d'entraînement (AUC-PR, précision, rappel, F1-score) n'étaient pas toutes disponibles dans la documentation du projet. Seul l'AUC-PR du XGBoost calibré est mentionné dans le script d'entraînement.
- **Dataset PaySim** : Le dataset PaySim est une simulation et ne capture pas la complexité des comportements de fraude réels. De plus, le script d'entraînement charge un sous-ensemble (1 million de lignes) du dataset complet.
- **Caractéristiques des données** : Les informations détaillées sur la distribution des variables, la corrélation entre features et les patterns de fraude spécifiques au contexte métier n'étaient pas documentées.

10.3.2 Limites fonctionnelles

- **Intégration BankMatch** : L'intégration avec la plateforme BankMatch est implémentée dans le code mais désactivée par défaut (`BANKMATCH_INTEGRATION_ENABLED=false`) en attente de la finalisation du contrat API par l'équipe BankMatch. Le flux complet n'a pas pu être validé de bout en bout.
- **Tests E2E** : Les tests d'intégration utilisent des mocks (respx) pour simuler le service Fraud Detection. Un véritable test end-to-end avec les services réels démarrés en Docker Compose n'a pas été réalisé.
- **Couverture de tests** : Le retry exponentiel n'est pas couvert par les tests automatisés. Certains scénarios d'erreur (timeouts, erreurs réseau spécifiques) ne sont pas testés.
- **CI/CD** : L'infrastructure CI/CD et les benchmarks de performance ne sont pas encore implémentés (marqués "en attente" dans le dépôt).

10.3.3 Limites techniques

- **Représentation des montants** : La détection de doublons repose sur une tolérance explicite de ± 0.02 EUR pour compenser l'utilisation de `float` au lieu de `Decimal`. Cette tolérance est fonctionnelle mais doit être configurable selon

le contexte client.

- **Retry exponentiel** : Le retry exponentiel est codé en dur (3 tentatives, délai initial 0.5s). Il pourrait être externalisé dans la configuration si le service était déployé plus largement.
- **Secrets JWT** : Selon `TESTING_FINDINGS_AND_RECOMMENDATIONS.md`, les secrets JWT actuels font 19-24 octets, en dessous des 32+ octets recommandés par la RFC 7518.
- **Parsing XML** : L'utilisation de `lxml` pour le parsing des fichiers CAMT.053 expose potentiellement à des attaques XXE (XML External Entity) si non configuré correctement.
- **Monitoring** : Grafana était non fonctionnel au moment du test (19 août 2026) et certaines fonctionnalités de monitoring sont marquées comme "recommandées" mais non pleinement intégrées.

10.3.4 Limites de périmètre

- Le périmètre de la plateforme BankMatch et les détails de l'orchestration centrale ne sont pas entièrement couverts dans ce rapport.
- Les modules de trésorerie, de reporting et d'assistance financière (AI Financial Copilot) n'étaient pas dans mon périmètre technique.
- Le rapprochement bancaire proprement dit relève d'un périmètre plus large que celui de ce stage.

10.4 Perspectives d'amélioration

Pour une future évolution du système, plusieurs améliorations sont envisageables :

- ajouter une persistance des statistiques d'upload et des résultats de validation ;
- intégrer des tests end-to-end avec les services réels démarrés en Docker Compose ;
- rendre configurable la tolérance de doublons et les délais de retry ;
- compléter la couverture des formats et documenter plus précisément les spécifications d'API côté BankMatch ;
- renforcer la sécurité des flux d'entrée et la gestion des secrets de configuration.

Les compétences techniques acquises incluent le parsing de formats bancaires, la validation de données avec tolérances, l'authentification JWT inter-services, les tests d'intégration et la documentation technique. Ce travail m'a fait passer de la théorie à la pratique et comprendre les contraintes réelles d'un système financier orienté données et sécurité.

Chapitre 11

Références

Documentation technique

- 1 **Tiangolo, S.** *FastAPI Documentation*. 2026. <https://fastapi.tiangolo.com/> [Consulté le 24/09/2026]
- 2 **Google.** *Angular Documentation*. 2026. <https://angular.io/> [Consulté le 24/09/2026]
- 3 **Docker Inc..** *Docker Documentation*. 2026. <https://docs.docker.com/> [Consulté le 24/09/2026]
- 4 **Neo4j.** *Neo4j Documentation*. 2026. <https://neo4j.com/docs/> [Consulté le 24/09/2026]
- 5 **Supabase.** *Supabase Documentation*. 2026. <https://supabase.com/docs> [Consulté le 24/09/2026]
- 6 **Lundberg, S. M., & Lee, S. I.** *A Unified Approach to Interpreting Model Predictions*. NIPS 2017. <https://shap.readthedocs.io/en/latest/> [Consulté le 24/09/2026]
- 7 **SWIFT.** *MT940 Standard*. <https://www.swift.com/standards/data-standards/swift-fin> [Consulté le 24/09/2026]
- 8 **ISO 20022.** *CAMT.053 Bank-to-Customer Account Statement*. <https://www.iso20022.org/catalogue-messages> [Consulté le 24/09/2026]
- 9 **Chen, T., & Guestrin, C.** *XGBoost : A Scalable Tree Boosting System*. KDD 2016. <https://xgboost.readthedocs.io/> [Consulté le 24/09/2026]
- 10 **Liu, F. T., Ting, K. M., & Zhou, Z. H.** *Isolation Forest*. ICDM 2008.
- 11 **lxml.** *lxml Documentation*. <https://lxml.de/> [Consulté le 24/09/2026]
- 12 **Jones, M., & Bradley, J.** *RFC 7519 : JSON Web Token (JWT)*. IETF 2015. <https://tools.ietf.org/html/rfc7519>
- 13 **Jones, M.** *RFC 7518 : JSON Web Algorithms (JWA)*. IETF 2015. <https://tools.ietf.org/html/rfc7518>
- 14 **Lopez, E. et al.** *PaySim : A Financial Mobile Money Simulator*. 2016. <https://github.com/EdgarLopezPhD/PaySim> [Consulté le 24/09/2026]
- 15 **Niculescu-Mizil, A., & Caruana, R.** *Predicting Good Probabilities with Su-*

- ervised Learning*. ICML 2005.
- 16 **Platt, J.** *Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods*. Advances in Large Margin Classifiers, 1999.
- 17 **scikit-learn**. *RandomForestClassifier Documentation*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> [Consulté le 24/09/2026]
- 18 **scikit-learn**. *CalibratedClassifierCV Documentation*. <https://scikit-learn.org/stable/modules/calibration.html> [Consulté le 24/09/2026]
- 19 **ISO 20022**. *PAIN.001 Customer Credit Transfer Initiation*. <https://www.iso20022.org/catalogue-messages> [Consulté le 24/09/2026]

Réglementation et normes

- 20 **European Parliament and Council**. *Directive (EU) 2015/2366 on payment services (PSD2)*. 2015. <https://eur-lex.europa.eu/eli/dir/2015/2366/oj> [Consulté le 24/09/2026]
- 21 **FATF**. *International Standards on Combating Money Laundering and the Financing of Terrorism*. 2024. <https://www.fatf-gafi.org/> [Consulté le 24/09/2026]

Documentation interne

- 22 **Projet interne**. *ARCHITECTURE_DECISIONS.md*. 2026. Spécifications des décisions d'architecture et statut d'implémentation.
- 23 **Projet interne**. *INTEGRATION.md*. 2026. Guide d'intégration et contrats API pour les modules Multi-Banking et Fraud Detection.
- 24 **Projet interne**. *TESTING_FINDINGS_AND_RECOMMENDATIONS.md*. 2026. Résultats des tests et recommandations d'amélioration.
- 25 **Projet interne**. *TESTING_REPORT.md*. 2026. Rapport détaillé des tests effectués sur l'application BankMatch.